

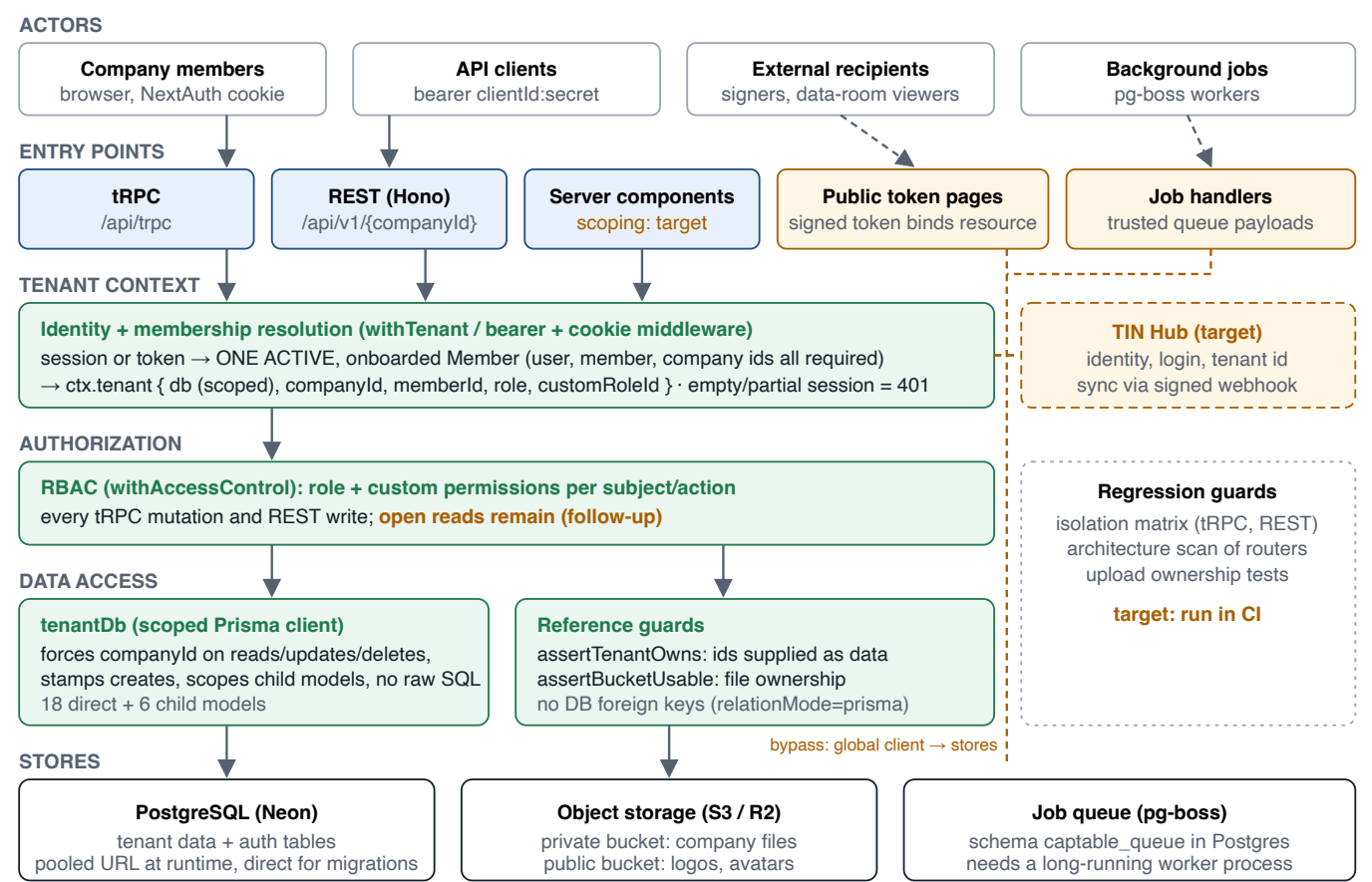
Captable Multitenancy: Architecture Blueprint

Target structure of a multi-company cap-table platform on Next.js, tRPC, Prisma and Neon, linked to the TIN Hub for identity. Version 1 · 2026-10-05 · derived from the code (graph of 534 files, 2,196 nodes, 90 communities) and checked against current library documentation.

BUILT exists in the code today and is covered by tests **TARGET** part of the architecture, not yet implemented. This document describes structure; the work sequence is in the separate plan.

1. Architecture at a glance

One application serves many companies. **A company is the tenant.** A person (User) joins companies through memberships (Member), each with a status and a role. Every request must resolve to exactly one active membership, and every tenant-owned row is reached only through a client scoped to that membership's company. Identity (who the person is) is meant to come from the Hub; authorization inside a company stays in Captable.



Dashed paths (public token pages, job handlers) skip the membership layer by design: the signed token or the trusted queue payload binds the resource. They use the global client and are the only code allowed to.

Figure 1. Layered architecture. Green = built and tested; orange = target or by-design exception.

2. Tenancy model

Entities

Entity	Meaning
Company	The tenant. Internal cuid id plus a unique 12-char publicId used in URLs and in storage keys.

Rules

- A user may belong to many companies; one membership is *active* per session.
- Only an **ACTIVE, onboarded** member can act. The JWT carries companyId / memberId as hints; they are re-verified against the database on every request.

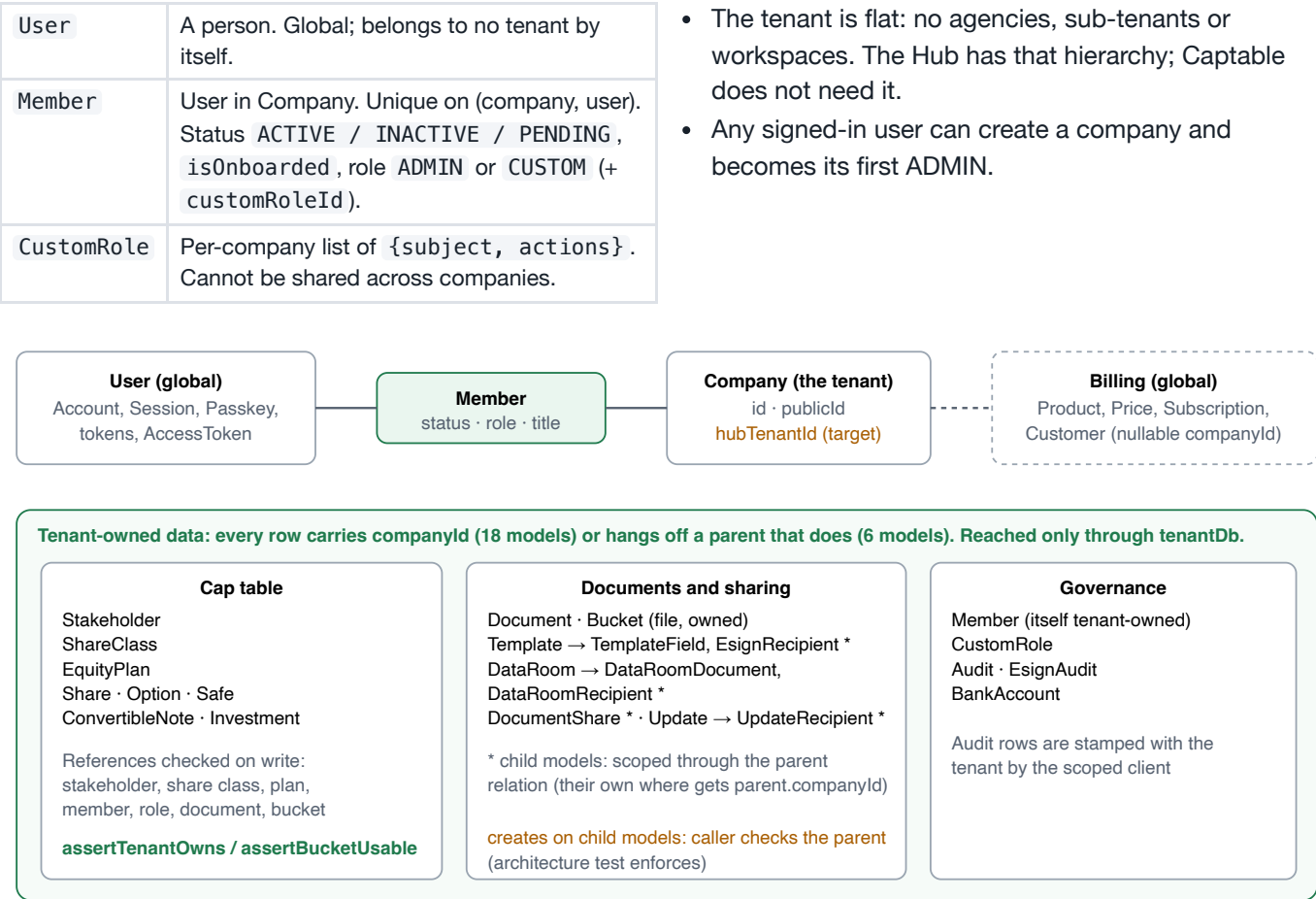


Figure 2. Data model by ownership. 37 models: 18 tenant-owned, 6 child, 13 global (including Company, User, authentication and billing).

3. Request lifecycle and trust boundaries

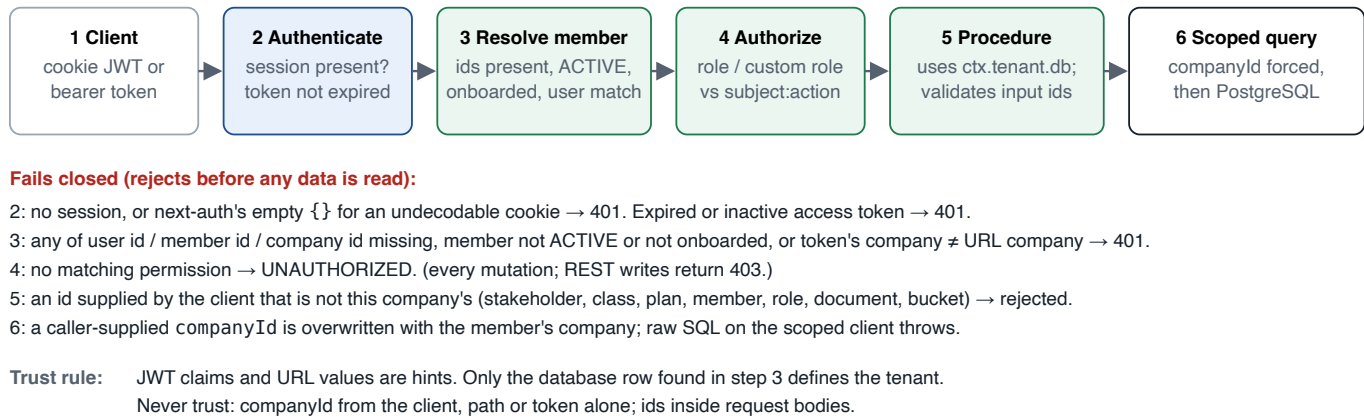


Figure 3. Request path (tRPC and REST share steps 2 to 6). Steps 3 and 6 are the structural guarantees.

4. Isolation guarantees

#	Invariant	Mechanism	Status
I1	A request never reads or changes another company's rows	tenantDb forces companyId (equality) on every where-based operation; child models scoped through their parent	BUILT
I2	A client cannot link its data to another company's rows	assertTenantOwns for stakeholder, class, plan, member, role, document; assertBucketUsable for files	BUILT

13	Files belong to one company	<code>Bucket.companyId</code> ; server-issued key <code>\${companyPublicId}/...</code> ; exact key-shape check on register; owner check on download	BUILT
14	Every entry point proves identity	Empty or partial sessions rejected in <code>checkMembership</code> and the cookie middleware; no unauthenticated server actions	BUILT
15	Public links reach only their own resource	Signed token binds template, recipient, data room or update; one-time signing state	BUILT · expiry TARGET
16	Inside a company, actions follow the member's role	RBAC subjects and actions on every mutation (tRPC and REST); role definition and assignment are ADMIN-only; last-ADMIN protection; least-privilege default role	BUILT (every tRPC mutation and every REST write; open reads remain, see below)
17	Tenant data is never read by unscoped code outside the by-design exceptions	Architecture scan of routers and REST routes; dashboard pages through one scoped helper	routers, REST and pages BUILT
18	These guarantees cannot silently regress	Two-tenant isolation matrix (tRPC and REST), upload tests, architecture test; run in CI	tests BUILT · CI TARGET

Known limits of the scoped client (design constraints, not defects)

- It does not cover **nested writes** (relation `connect` inside `data`), **relation includes**, **ids supplied as data** (guards cover those), or **creates on child models** (callers must check the parent).
- It rewrites any `companyId` condition to equality, so code that must read *another* company (listing the user's companies, switching company, accepting an invite) uses the global client on purpose and is allow-listed in the architecture test.
- Prisma treats an `undefined` filter value as "do nothing"; that is why identity fields are checked for presence before any lookup.

5. Authentication and session

Channel	Credential	Resolves to
Web app	NextAuth 4 JWT cookie (credentials with verified email, passkeys, Google). Secret must be 32+ chars. No automatic account linking by email.	Session with <code>id</code> , <code>memberId</code> , <code>companyId</code> , <code>companyPublicId</code> ; re-verified per request
REST, cookie	Same cookie, session fetched from <code>/api/auth/session</code>	Same member check; empty session rejected
REST, bearer	<code>clientId:secret</code> , secret stored as a hash; active, unexpired token	The token's user, as ACTIVE member of the company in the URL
Public links	Signed HS256 token (e-sign, data room, update)	One resource; no membership
Jobs	pg-boss payload written by the server	Company id supplied by the enqueuer

Tenant switching

The session points at the member with the most recent `lastAccessed`. `switchCompany` accepts only the caller's own ACTIVE, onboarded membership, updates `lastAccessed` and refreshes the session. It deliberately uses the global client because it reads a membership in a different company.

6. Authorization architecture BUILT

A permission is `subject:action`. ADMIN holds `*`; CUSTOM holds the list in its `CustomRole`; a member with no role holds nothing. Enforcement is a tRPC middleware (`withAccessControl`) driven by `.meta({policies})`, so a procedure's required permission is visible beside its definition.

Subjects: `billing`, `members`, `stakeholder`, `roles`, `audits`, `documents`, `company`, `developer`, `bank-accounts`, `securities`, `cap-table-settings`, `updates`, `data-rooms`, `templates`. The REST API enforces the same check (fail-closed) on stakeholder and share writes, so an API token carries only its owner's role in the company named in the URL.

Rule	Default
Read cap-table data	any active member (exceptions: stakeholders, documents, and anything that mints a public link need their read permission)
Create, edit, delete securities and settings	ADMIN or granted
Define or assign roles; invite with a role; act on an ADMIN	ADMIN only (an unchanged role resent by the UI is not a change)
Other member management (titles, non-admin members)	ADMIN or granted <code>members</code> permission
Last ADMIN	cannot be removed, deactivated or demoted
New member without a role	no permissions (the schema default is removed)

7. File storage architecture



Download: `bucket.getUrl` → look up the bucket through the scoped client (owner check) → 10-minute signed GET URL. Unowned or foreign bucket: not found.

Public bucket: logos and avatars only; images only (PNG, JPEG, WebP, GIF); key under the signed-in user's id.

Never exposed: any callable that signs or deletes by key without a session and an ownership check (the old server-action module is gone).

Figure 4. Upload and download path. The client never chooses the storage prefix.

8. Hub integration TARGET

Principle: the Hub owns identity, Captable owns authorization inside a company, and tenant logic is never duplicated. Captable keeps its own `Company / Member` model and *links* to the Hub; it does not adopt the Hub's packages, because they depend on Supabase's `auth.uid()` for isolation and give a Prisma app no protection.

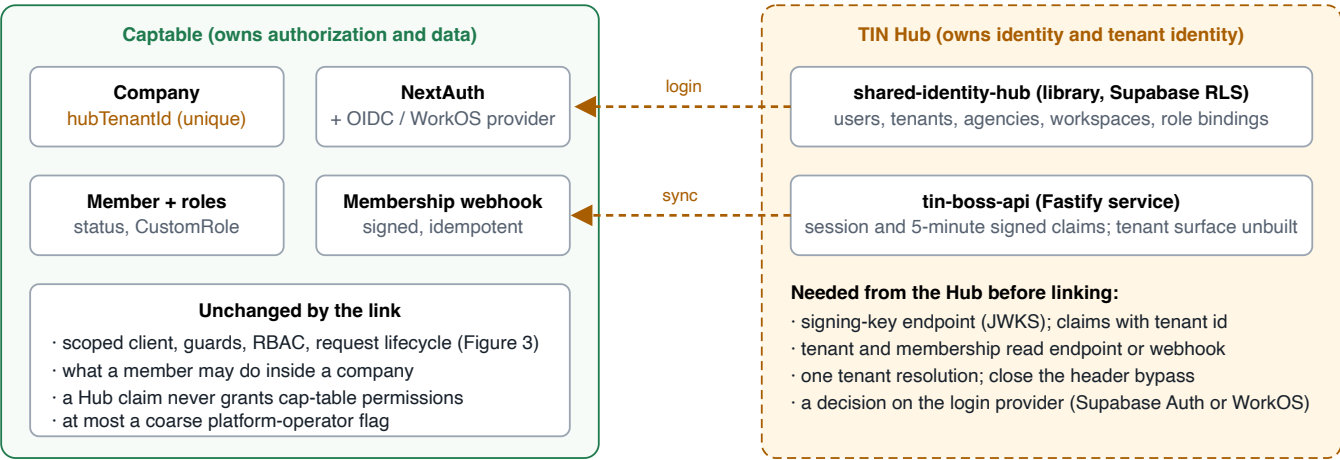


Figure 5. Target integration. Dashed = not yet buildable; the Hub side does not provide the required endpoints today.

Concern	Owner
Who a person is, login, platform operators	Hub
Which tenant a customer is (<code>tenant_id</code>)	Hub; stored on <code>Company.hubTenantId</code>
Membership, status, role inside a company	Captable (<code>Member</code>); status may be pushed by the Hub
What a role may do on cap-table data	Captable (RBAC)
Tenant-scoped data access	Captable (scoped client)

9. Deployment topology

Component	Choice	Why
Application	Long-running Node server (recommended: Fly.io; Railway equivalent)	The job queue (<code>pg-boss</code>) needs a process that stays up; serverless hosting would need the jobs moved to an external queue
Database	Neon PostgreSQL. Pooled URL for the app at runtime on serverless; direct URL for migrations and <code>pg-boss</code>	Prisma migrations and queue locks do not work through a pooler
Object storage	Cloudflare R2 or S3, private and public buckets	The app already speaks the S3 API
Email	SMTP provider (Resend or Postmark)	Invites, e-sign requests
Secrets	<code>NEXTAUTH_SECRET</code> (32+ chars; also signs public-link tokens), storage and SMTP credentials	The app refuses to start with a short secret
Quality gate	CI job: Postgres service, migrate, tests, typecheck, lint TARGET	No workflow runs tests today

10. Threat model summary

Threat	Where it appears	Control	Residual
Spoofing a member	Cookie and bearer paths	Per-request member lookup; empty-session rejection; hashed token secret with expiry	Public links expire after 30 days; links sent before expiry existed keep working until the operator enables the strict switch

Cross-tenant read or write	Any query; ids in request bodies	Scoped client; reference guards; two-tenant matrix	Nested writes and includes (documented limits); dashboard pages use the scoped helper
Elevation inside a tenant	Mutations without role checks; self-granting roles; default admin role	RBAC on tRPC and REST; ADMIN-only role management; least-privilege default; last-admin rule	Many reads still open to any active member by design (cap-table lists); the last-admin check runs serializable with one retry
File access	Presigned URLs, storage keys	Server-built keys, owner check, exact key shape, image-only public uploads	Size and content type are signed (25 MiB private, 5 MiB public images, image types only for public uploads); to be confirmed with one real upload against the storage backend; legacy unowned buckets unusable until assigned
Information disclosure	Errors, global unique emails	Not-found instead of forbidden for foreign ids	Stakeholder email is unique per company; a duplicate within one company still returns a generic 500 on REST
Regression	New procedure bypassing the structure	Architecture test; coverage guard in the matrix	Not enforced in CI TARGET

11. Architecture decisions

Decision	Reason	Revisit when
Company is the tenant; flat, no hierarchy	Matches the product; the Hub's agency/workspace tiers add cost with no Captable use	A customer needs sub-organisations
Application-level scoping (client extension), not database row-level security	Works with Prisma and Neon today; small blast radius; testable	A second isolation bug despite it, or a compliance requirement (then add RLS using per-query <code>set_config</code> , a documented Prisma pattern)
Keep NextAuth and Prisma; link to the Hub by tenant id	The Hub's packages require Supabase; adopting them is a rewrite	The Hub ships a framework-neutral adapter
Buckets are owned by a company; keys are server-issued	Object ownership cannot be inferred from a URL; client-chosen keys were the hole	Per-file sharing across companies is needed
Public flows use signed tokens, not accounts	Signers and data-room viewers are not members	Tokens need revocation (add a token table)
Authorization stays in Captable	Roles are about cap-table actions the Hub does not model	The Hub gains a per-app permission service

Evidence: code graph of `src/` (graphify), library behaviour checked with Context7 (Prisma `undefined` filters, query extensions and per-query `set_config`; Next.js server actions as public endpoints; NextAuth empty session; tRPC context middleware) and by the automated tests (193 passing, production build green).